Grundlagen der IT-Sicherheit Praktikum

SoSe 26

Prof. R. Zahoransky

Hochschule Furtwangen (University of Applied Science)

Aufgabenblatt 3 (Brute Force am Beispiel einer Vigenère-Chiffre)

Lernziele: Verständnis für Kryptologie, Schlüsselräume und Brute Force-Angriffe

Abgabetermin wird in FELIX bekannt gegeben

Aufgabe 1

Sie möchten Nachrichten verschlüsselt zwischen Ihnen und den Kommilitonen austauschen. Dabei folgen Sie **nicht** dem Rat, etablierte und bekannte Verschlüsselungsverfahren zu nutzen. Stattdessen wollen Sie ein eigenes Verfahren anwenden:

Anbei finden Sie Java-Methoden, um eine Vignere-Chiffre zu implementieren. Eclipse ist auf dem Praktikum-Image installiert. Damit können Sie die Java-Methoden nutzen. Erzeugen Sie Ihre verschlüsselten Texte mit diesem Beispiel. Gerne können Sie das Beispiel auch in andere Sprachen portieren. Nutzen Sie keine Sonderzeichen, außer Leerzeichen und Punkte. Alle Zeichen sollen Großbuchstaben sein.

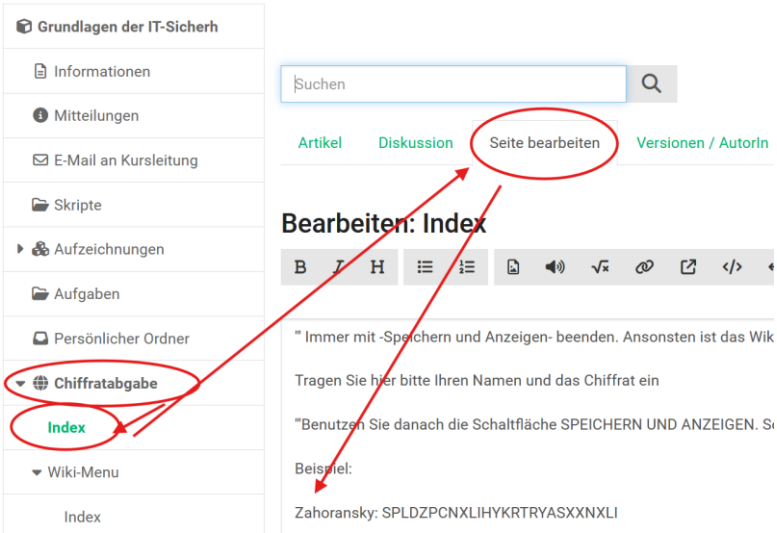
Beispiel: Es wird der Text „Studieren an der HFU ist toll.“ mit dem Schlüssel „toll“ verschlüsselt:

```
STUDIEREN AN DER HFU IST TOLL.
TOLLTOLLTOLLTOLLTOLLTOLLTOLLTO
```

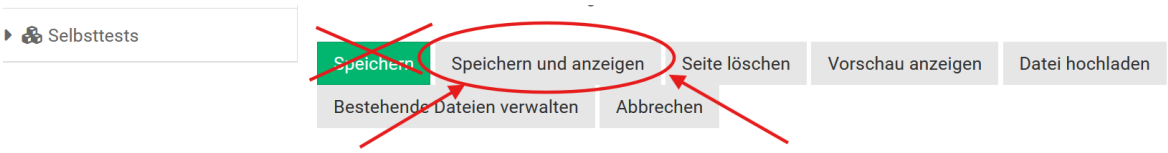
wird zu

```
LHFOBSCPG LY RPC VQF WDE HZWE.
```

Verschlüsseln Sie einen deutschen Text (**mindestens einen Satz**) mit einem **vierstelligen** Schlüssel. Verteilen Sie Ihre Nachrichten an alle. Speichern Sie hierfür Ihre Nachrichten (mit Ihrem Namen) in Felix ab. Nutzen Sie das Wiki, damit alle darauf Zugriff haben. Benutzen Sie den Button Das ist wichtig für die nächste Aufgabe. Es folgt eine bebilderte Anleitung:



NUTZEN DIE FUNKTION „SPEICHERN UND ANZEIGEN“. SONST WIRD DAS WIKI GESPERRT



Aufgabe 2

Wie viele der Texte Ihrer Kommilitonen können Sie entschlüsseln? Erstellen Sie hierfür eine Methode, die alle möglichen Schlüsselkombinationen eines viertstelligen Schlüssels durchläuft und für jeden Schlüssel prüft, ob es der korrekte Schlüssel ist.

Hinweis: Sie finden unten vorgefertigte Methode, um an Hand von statistischen Merkmalen zu erkennen, ob ein gegebener String ein deutscher Text sein könnte. Diese Methode funktioniert nicht perfekt. Sie müssen Ihre Ergebnisse eventuell manuell filtern oder die threshold-Variable anpassen.

Aufgabe 3

Wieso war es Ihnen in diesem Beispiel möglich die verschlüsselten Texte zu entschlüsseln?

1. Wie groß ist der Schlüsselraum? Das bedeutet wie viele möglich Schlüssel gibt es?
2. Was wäre, wenn der Schlüssel doppelt so viele Stellen hätte, also acht Stellen?
3. Könnten Sie einen achtstelligen Schlüssel immer noch brechen?
4. Wie groß wäre der Schlüsselraum bei einem vierstelligen Schlüssel, der nicht nur aus A-Z, sondern aus {a-z, A-Z} besteht, also aus doppelt so vielen Zeichen?
5. Argumentieren Sie, was die Sicherheit eines Schlüssels mehr verbessern würde: Ein größeres Alphabet (d.h. mehr Möglichkeiten pro Stelle) oder mehr Stellen. Wie berechnen Sie die Anzahl möglicher Permutationen? Die Änderung welches Parameters lässt die Anzahl der Permutationen schneller steigen?
6. Wieso kann eine EC-Karte dennoch (einigermaßen) sicher angesehen werden, obwohl eine PIN lediglich vierstellig und aus den Zeichen 0-9 aufgebaut ist?

Methoden zum Ver- und Entschlüsseln und Testen der Lesbarkeit. Zu speichern als `VignereChiffreStatic.java`:

```java
import java.util.Arrays; import java.util.HashSet;

public class VignereChiffreStatic {

	static String[] keywords = { "die", "der", "und", "in", "zu", "den", "das", "nicht", "von", "sie", "ist", "des",
	                             "sich", "mit", "dem", "dass", "er", "es", "ein", "ich", "auf", "so", "eine", "auch", "als", "an", "nach",
	                             "wie", "im", "für", "fuer", "fur" };
	static HashSet<String> germanKeywordsSet = new HashSet<String>();
	static double threshold = 0.3; // Prozentsatz der Wörter, die erkannt werden müssen

	public static void main(String[] args) {
		String s = "Studieren an der HFU ist toll.";
		String key = "TOLL";
		System.out.println("Testen der Verschlüsselung mit: " + s + " und Schlüssel: " + key);
		String encrypted = encrypt(s, "TOLL");
		System.out.println(s);
		System.out.println(encrypted + " (verschlüsselt)");
		System.out.println(decrypt(encrypted, "TOLL") + " (entschlüsselt)");
		System.out.println();
		System.out.println("Ist der Text (vermutlich) lesbar?: " + isProbablyReadable(s));
		System.out.println();
		System.out.println("Nun geht es an die Aufgabe: Versuchen Sie alle möglichen Schlüssel. Erweitern Sie den Code unten");

		// Hinweis: Permutationen der Länge zwei aus der Menge {A,B,C}
		String[] buchstaben = { "A", "B", "C" };
		for (String buchstabe1 : buchstaben) {
			for (String buchstabe2 : buchstaben) {
				System.out.println("Hier könnten Sie mit dem Schlüssel " + buchstabe1 + buchstabe2 + " testen");
				// Testen Sie: Ist der entschlüsselte Text lebar? Wenn ja: Geben Sie Schlüssel und Text aus.
			}
		}

	}

	/**
	 * String s verschlüsseln
	 * @param text             Nachricht im Klartext
	 * @param keyForEncryption Schlüssel
	 * @return Verschlüsselte Nachricht
	 */
	public static String encrypt(String text, String keyForEncryption) {
		text = text.toUpperCase(); // alles Grossbuchstaben
		char[] chars = text.toCharArray();
		char[] key = keyForEncryption.toUpperCase().toCharArray();
		int keyLength = key.length;
		for (int i = 0; i < chars.length; i++) {
			char charToShift = key[i % keyLength];
			chars[i] = shift(chars[i], charToShift); // Sonderzeichen ignorieren. Schlüsselindex weiterzählen
		}
		return String.valueOf(chars); // wieder in String wandeln
	}

	/**
	 * String s entschlüsseln
	 * @param text             Nachricht als Chiffrat
	 * @param keyForDecryption Schlüssel
	 * @return Entschlüsselte Nachricht
	 */
	public static String decrypt(String text, String keyForDecryption) {
		text = text.toUpperCase(); // alles Grossbuchstaben
		char[] chars = text.toCharArray();
		char[] key = keyForDecryption.toUpperCase().toCharArray();
		int keyLength = key.length;
		for (int i = 0; i < text.length(); i++) {
			char charToShift = key[i % keyLength];
			chars[i] = deShift(chars[i], charToShift);
		}
		// wieder in String wandeln
		return String.valueOf(chars);
	}

	private static char shift(char ch, char k) {
		if (ch >= 'A' && ch <= 'Z') { // Sonderzeichen nicht behandeln
			return (char) (((ch - 'A') + (k - 'A')) % 26 + 'A');
		} else {
			return ch;
		}
	}

	private static char deShift(char ch, char k) {
		if (ch >= 'A' && ch <= 'Z') { // Sonderzeichen nicht behandeln
			return (char) (((ch - 'A') - (k - 'A') + 26) % 26 + 'A');
		} else {
			return ch;
		}
	}

	/**
	 * Wie häufig kommt einer der in keyword definierten Wörter im Text vor?
	 * @param s Der Text
	 * @return Anzahl der Übereinstimmungen
	 */
	protected static int occurencesOfKeywords(String s) {
		if (germanKeywordsSet.isEmpty()) // Array in Set überführen
			germanKeywordsSet.addAll(Arrays.asList(keywords)); // sofern nicht bereits geschehen
		String[] words = s.toLowerCase().replaceAll("\\.", "").split(" ");
		int hits = 0;
		for (String word : words) {
			if (germanKeywordsSet.contains(word))
				hits++;
		}
		return hits;
	}

	/**
	 * Ist der Text lesbar?
	 * @param s Der Text zum Testen
	 * @return wahr, wenn der Text vermutlich lesbar ist
	 */
	public static boolean isProbablyReadable(String s) {
		int hits = occurencesOfKeywords(s);
		int word = s.toLowerCase().replaceAll("\\.", "").split(" ").length;
		double percentage = ((double) hits) / word;
		return percentage >= threshold;
	}

}
```

Erweitern Sie diesen Code, um für alle vierstelligen Schlüssel aus A-Z die verschlüsselten Texte aus Felix zu entschlüsseln. Testen Sie mit der Methode **`isProbablyReadable`**, ob das Ergebnis lesbar ist. Falls ja, lassen Sie sich Schlüssel und Text ausgeben.